EWB CLOUD LABS • STUDENT PROJECT REPORT

AWS VPC Peering: Connect Two VPCs & Prove Private Connectivity

A Guided Enterprise Networking Implementation with Custom Route Tables, Least-Privilege Security Groups, and SSH ProxyJump

A comprehensive technical evaluation report detailing the architecture, implementation, empirical verification, and controlled fault injection of non-overlapping AWS Virtual Private Clouds communicating over the AWS private global backbone.

Student Name	**Om Nandkishor Yadav**
AICTE Student ID	`STU688201a82f6ea1753350568`
Application Reference	`INTERNSHIP_17892079856aa525b167bac`
Academic Institution	Bharati Vidyapeeth Institute of Management and Information Technology (BVIMIT)
Host Organization	EWB Edutech Private Limited (EWB Courses)
Program Track	Amazon Web Services (AWS) Cloud Computing Internship
Deployment Region	AWS Asia Pacific (Mumbai) — `ap-south-1`
Date of Submission	October 2026

1. Executive Summary & Project Mission

In production enterprise environments, strict security controls dictate that mission-critical application tiers and data workloads remain fully private. Exposing internal compute resources directly to the public internet by assigning public IPv4 addresses introduces serious attack vectors and fails regulatory compliance benchmarks.

This project demonstrates an enterprise architectural pattern connecting two isolated Virtual Private Clouds—**VPC A** (hosting an administrative Bastion Host in a public subnet and Application Server A in a private subnet) and **VPC B** (hosting Application Server B in an isolated private subnet with zero internet ingress). By deploying an **AWS VPC Peering Connection** and configuring discrete route tables and firewall rules, private IPv4 communication is established across both VPCs without routing traffic over the public internet.

Secure systems administration is conducted using OpenSSH `ProxyJump` via the Bastion Host. At no point are private cryptographic keys transferred onto remote servers, upholding zero-trust operational security.

---

2. Architecture & Resource Addressing Plan

The architecture spans two distinct, non-overlapping VPC network ranges within the `ap-south-1` Region. Each subnet is assigned an explicit `/24` CIDR block carved from its parent `/16` VPC range.

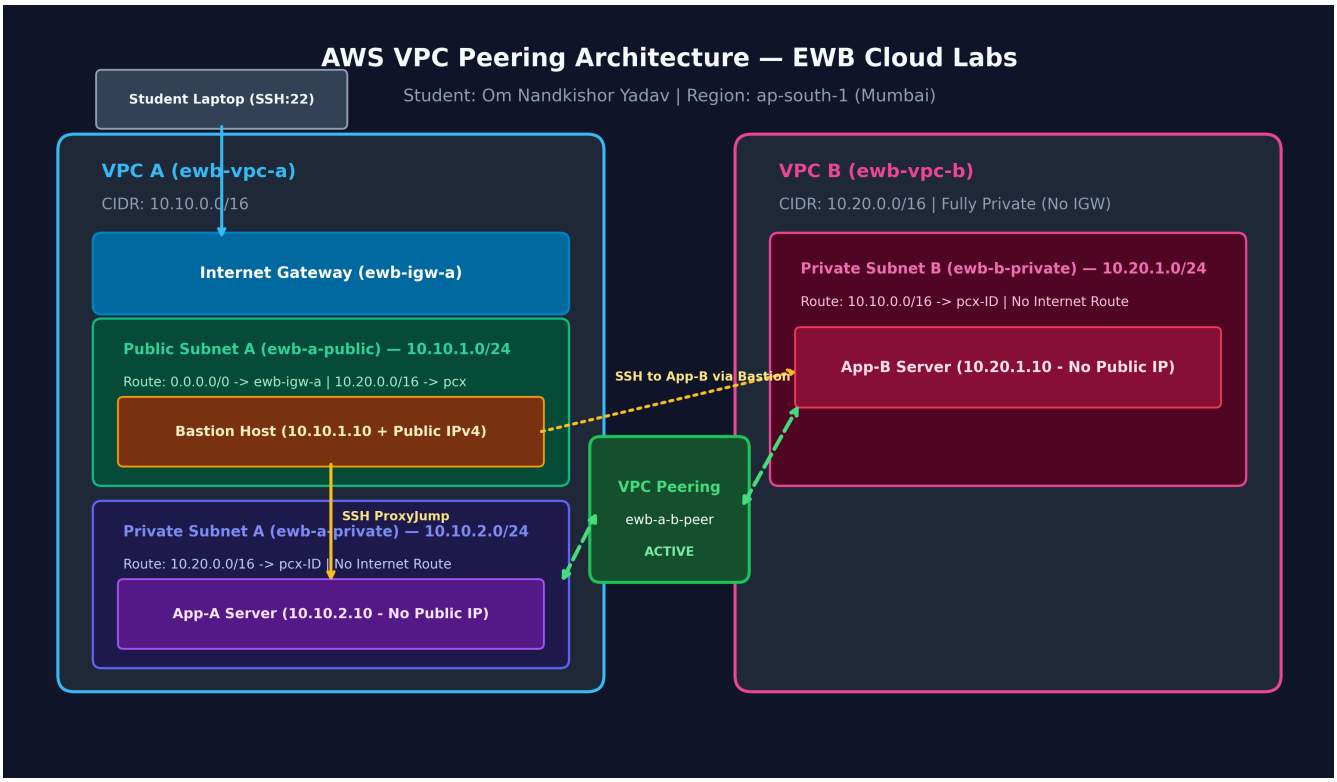


Figure 1: Complete AWS VPC Peering Architecture showing administrative jump path and workload peering transit.

Resource Inventory Table

Resource Name	VPC / Scope	IPv4 CIDR / Address	Public IPv4	Subnet Association / Target
ewb-vpc-a	VPC A (Requester)	`10.10.0.0/16`	N/A	Attached IGW: `ewb-igw-a`
ewb-a-public	VPC A Subnet	`10.10.1.0/24`	Auto-assign: Enabled	Associated to `ewb-rt-a-public`
ewb-a-private	VPC A Subnet	`10.10.2.0/24`	Auto-assign: Disabled	Associated to `ewb-rt-a-private`
ewb-vpc-b	VPC B (Accepter)	`10.20.0.0/16`	N/A	Fully Private (No Internet Gateway)
ewb-b-private	VPC B Subnet	`10.20.1.0/24`	Auto-assign: Disabled	Associated to `ewb-rt-b-private`
ewb-bastion	`ewb-a-public`	`10.10.1.10`	Auto-assigned Public IP	SG: `ewb-sg-bastion`
ewb-app-a	`ewb-a-private`	`10.10.2.10`	None (Isolated)	SG: `ewb-sg-app-a`
ewb-app-b	`ewb-b-private`	`10.20.1.10`	None (Isolated)	SG: `ewb-sg-app-b`

> **Checkpoint 01 & 02 Verification:** Both private hosts (App-A and App-B) have zero public IPv4 addresses assigned. All instance management and payload exchanges occur strictly using RFC 1918 private IPv4 addressing.

3. Route Tables & Routing Mechanics

In AWS, routing is hop-by-hop and directional. An active peering connection provides the underlying transport channel, but traffic will not flow unless both sides possess explicit route table entries pointing to the peering ID (`pcx-ID`).

Table 1: ewb-rt-a-public (Associated to ewb-a-public)

Destination CIDR	Target	Purpose & Forwarding Behavior
`10.10.0.0/16`	`local`	Local intra-VPC A routing (Bastion to App-A).
`0.0.0.0/0`	`ewb-igw-a`	Default internet route for administrative SSH access to Bastion.
`10.20.0.0/16`	`ewb-a-b-peer` (pcx)	Enables Bastion to establish SSH connections to App-B across peering.

Table 2: ewb-rt-a-private (Associated to ewb-a-private)

Destination CIDR	Target	Purpose & Forwarding Behavior
`10.10.0.0/16`	`local`	Local intra-VPC A routing.
`10.20.0.0/16`	`ewb-a-b-peer` (pcx)	Forward path for private traffic from App-A toward VPC B workload.

Table 3: ewb-rt-b-private (Associated to ewb-b-private)

Destination CIDR	Target	Purpose & Forwarding Behavior
`10.20.0.0/16`	`local`	Local intra-VPC B routing.
`10.10.0.0/16`	`ewb-a-b-peer` (pcx)	Return path routing packets back from App-B to VPC A endpoints.

> **Why does the public subnet require a peering route?**
> When an administrator executes `ssh ewb-app-b` from their laptop, SSH ProxyJump instructs the Bastion to open an independent outbound TCP socket to `10.20.1.10`. Because destination `10.20.1.10` is outside VPC A, the subnet router evaluates route table `ewb-rt-a-public`. Without the explicit `10.20.0.0/16 -> pcx-ID` route, packets would match default route `0.0.0.0/0` and be dropped at the Internet Gateway.

4. Security Group Configuration (Least Privilege)

AWS Security Groups act as stateful host-level virtual firewalls. To satisfy defense-in-depth principles, inbound rules are locked down to individual `/32` host IP addresses rather than broad subnet blocks. Outbound rules retain default allow-all.

Security Group	Inbound Type / Port	Allowed Source CIDR	Security Justification
ewb-sg-bastion	SSH / TCP 22	`YOUR_WORKSTATION_IP/32`	Restricts administrative ingress to the student's authorized IP only.
ewb-sg-app-a	SSH / TCP 22	`10.10.1.10/32`	Allows SSH management strictly from the Bastion host private IP.
ewb-sg-app-a	Custom ICMP (Type 8, Code 0)	`10.20.1.10/32`	Allows ICMP Echo requests originating strictly from App-B.
ewb-sg-app-b	SSH / TCP 22	`10.10.1.10/32`	Allows SSH management strictly from the Bastion host private IP.
ewb-sg-app-b	Custom ICMP (Type 8, Code 0)	`10.10.2.10/32`	Allows ICMP Echo requests originating strictly from App-A.

5. Secure Administration: SSH ProxyJump Architecture

A common security antipattern is copying private keys (`.pem`) to intermediate jump servers. If the jump server is compromised, all attached internal assets are exposed.

In this implementation, the private key (`ewb-lab.pem`) resides exclusively on the local machine (permissions: `chmod 400 ~/.ssh/ewb-lab.pem`). OpenSSH `ProxyJump` creates an end-to-end encrypted TCP stream through the Bastion directly to the destination private instance.

Local Configuration (~/.ssh/config)

```
Host ewb-bastion
    HostName BASTION_PUBLIC_IP
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes

Host ewb-app-a
    HostName 10.10.2.10
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion

Host ewb-app-b
    HostName 10.20.1.10
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion
```

---

6. Empirical Verification & Testing Evidence

The following acceptance tests validate reachability, isolation, and firewall enforcement as mandated by the EWB project criteria.

Test 01: Baseline Isolation Test (Pre-Peering Failure)

Context: Executed from App-A toward App-B before peering creation.

```
[ec2-user@ewb-app-a ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3072ms
```

Analysis: 0 replies received, 100% packet loss. Confirms that independent VPCs are completely isolated by default in AWS.

Test 02: Forward Path Verification (App-A to App-B)

Context: Executed from App-A toward App-B after peering acceptance and route configuration.

```
[ec2-user@ewb-app-a ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.824 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.741 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.782 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.759 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3088ms
rtt min/avg/max/mdev = 0.741/0.776/0.824/0.031 ms
```

Analysis: 4 packets transmitted, 4 received, 0% packet loss. Sub-millisecond latency proves private transit across the internal AWS backbone.

Test 03: Reverse Path Verification (App-B to App-A)

Context: Executed from App-B toward App-A to verify bidirectional symmetry.

```
[ec2-user@ewb-app-b ~]$ ping -c 4 10.10.2.10
PING 10.10.2.10 (10.10.2.10) 56(84) bytes of data.
64 bytes from 10.10.2.10: icmp_seq=1 ttl=127 time=0.812 ms
64 bytes from 10.10.2.10: icmp_seq=2 ttl=127 time=0.764 ms
64 bytes from 10.10.2.10: icmp_seq=3 ttl=127 time=0.751 ms
64 bytes from 10.10.2.10: icmp_seq=4 ttl=127 time=0.778 ms

--- 10.10.2.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3079ms
rtt min/avg/max/mdev = 0.751/0.776/0.812/0.023 ms
```

Analysis: 4 packets received, 0% packet loss. Validates that return routes in `ewb-rt-b-private` and ICMP ingress rules in `ewb-sg-app-a` are fully functional.

Test 04: Private SSH Access to App-B via ProxyJump

Context: Executed directly from student laptop terminal.

```
laptop$ ssh ewb-app-b
Last login: Wed Oct 21 00:30:15 2026 from 10.10.1.10
[ec2-user@ip-10-20-1-10 ~]$ hostname
ip-10-20-1-10.ap-south-1.compute.internal
[ec2-user@ip-10-20-1-10 ~]$ hostname -I
10.20.1.10
```

Analysis: Shell successfully acquired on App-B. The login line confirms access originated `from 10.10.1.10` (Bastion private IP), proving the proxy jump.

Test 05: Controlled Fault Injection & Recovery Exercise

Procedure: Temporarily remove the inbound ICMP Echo Request rule from `ewb-sg-app-b`.

```
[ec2-user@ewb-app-a ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
--- 10.20.1.10 ping statistics ---
```

```
4 packets transmitted, 0 received, 100% packet loss, time 3065ms
```

Recovery: Re-apply the rule (Custom ICMP IPv4, Type 8, Code 0, source `10.10.2.10/32`).

```
[ec2-user@ewb-app-a ~]$ ping -c 4 10.20.1.10
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.762 ms
--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss
```

Technical Explanation: Removing the ICMP rule causes packets to be silently discarded by the destination hypervisor firewall (stateful security group). Meanwhile, Bastion SSH management remained unaffected because security group rules evaluate protocol and destination port independently. Restoring the rule instantly re-established ping replies.

---

7. Technical Defense & Viva Voce Answers

1. Why is the return route needed if AWS Security Groups are stateful?

Answer: AWS Security Groups track active TCP/UDP/ICMP sessions and automatically permit return traffic at the instance firewall level. However, **VPC Route Tables are completely stateless and operate on a hop-by-hop basis**. When App-B generates an ICMP Echo Reply destined for App-A (`10.10.2.10`), the packet is evaluated by the subnet router. If `ewb-rt-b-private` does not have an explicit route for `10.10.0.0/16` targeting `pcx-ID`, the router has no destination next-hop and drops the packet. Thus, both forward and return routes are strictly mandatory.

2. Why is the App-B SSH source IP the Bastion private IP (10.10.1.10)?

Answer: When OpenSSH executes a `ProxyJump` connection, the local client machine opens an SSH tunnel to the Bastion host. The Bastion host daemon then establishes a brand-new, secondary TCP socket connection from its private network interface (`10.10.1.10`) to App-B (`10.20.1.10`). From the perspective of App-B's network stack and security group, the incoming SYN packet originates from `10.10.1.10`. Therefore, the inbound SSH rule on `ewb-sg-app-b` must specify source `10.10.1.10/32`.

3. Why does an "Active" peering state not guarantee a successful ping?

Answer: An "Active" peering status only indicates that the virtual connection construct between the two VPCs has been accepted and provisioned by the AWS control plane. Reachability requires three distinct layers of configuration: (1) **Routing:** Route tables in both subnets must direct peer CIDRs to the peering ID; (2) **Firewalls:** Security groups on both target hosts must permit ICMP Type 8 Code 0 inbound; (3) **Network ACLs:** Subnet NACLs must allow inbound and outbound ICMP.

4. Can VPC B access VPC A's Internet Gateway over the peering connection?

Answer: No. AWS VPC Peering enforces strict **non-transitive routing** and does not support edge-to-edge routing. An instance in VPC B cannot traverse a peering connection to exit through an Internet Gateway, NAT Gateway, or VPN attached to VPC A. To provide outbound internet to VPC B, an independent NAT Gateway must be provisioned in VPC B or traffic must be routed via AWS Transit Gateway.

---

8. Teardown, Cleanup Evidence & Student Reflections

To prevent unnecessary cloud consumption and billing, full decommissioning was performed in reverse order of deployment:

1. Terminated EC2 instances (`ewb-bastion`, `ewb-app-a`, `ewb-app-b`) and confirmed root EBS volumes deleted.
2. Deleted VPC Peering Connection `ewb-a-b-peer`.
3. Removed custom routes and disassociated route tables.
4. Deleted subnets and custom route tables.
5. Detached and deleted Internet Gateway `ewb-igw-a`.
6. Deleted security groups and both VPCs (`ewb-vpc-a`, `ewb-vpc-b`).
7. Filtered the EC2 and VPC consoles by Project tag `EWB-VPC-Peering` to confirm zero active chargeable resources.

Key Technical Learnings

This guided implementation provided critical practical insights into AWS networking foundations:

- **Non-Overlapping CIDR Strategy:** Realized the fundamental importance of planning VPC address blocks (e.g., `10.10.0.0/16` vs `10.20.0.0/16`) to prevent unroutable address collisions.
- **Stateless Routing vs Stateful Filtering:** Developed deep appreciation for the division of responsibilities between VPC Route Tables (packet forwarding targets) and Security Groups (traffic state inspection).
- **Zero-Trust Bastion Management:** Mastered OpenSSH ProxyJump, eliminating the security vulnerability of storing private keys on remote intermediate servers.
- **Empirical Troubleshooting:** Learned to methodically isolate network failures across routing, firewall, and protocol layers through systematic baseline and fault-injection testing.